复现操作步骤

作品：香山昆明湖 V2 自定义向量点积指令 vdot

参赛队伍：啊对对对对

代码仓库：https://github.com/cc77-builder/riscv/tree/release/cie2026-v1

本文按“快速检查 → 环境准备 → 阶段一 → 最终硬件 → 阶段二 → 阶段三”的顺序给出可直接执行的命令。所有命令都在仓库根目录执行，`$OUT` 是你新建的输出目录，所有运行结果都写到这里。

部分	内容	首次用时	需要
1	获取代码	10 ~ 20 分钟	git、git-lfs
2	快速检查：清单、模型、编码、日志	5 分钟	Python 3.10+
3	环境准备	1 ~ 2 小时	Ubuntu 22.04 / WSL2，内存 16 GB 以上
4	阶段一：hello 与向量加法	30 分钟	第 3 部分
5	构建最终硬件与 vdot NEMU	40 ~ 60 分钟	第 3 部分
6	阶段二：算术、译码、FU 与整核基础测试	30 分钟	第 5 部分
7	阶段三：微基准、软件库与波形	1 小时	第 5 部分

1. 获取代码

在 Linux 原生文件系统中执行（WSL 下不要放在 `/mnt/c`）。需要完整克隆：环境检查会核对固定的香山基线提交是否在当前历史中。

```
git clone --recurse-submodules -b release/cie2026-v1 https://github.com/cc77-builder/riscv.git
cd riscv
git lfs pull                       # 演示视频与全核 FST 波形
export OUT=$HOME/vdot-repro        # 新建的输出目录，后面所有结果都写到这里
mkdir -p "$OUT"
```

2. 快速检查（不需要硬件工具链）

```
python3 -B submission/verify.py
bash submission/reproduce.sh check
bash submission/reproduce.sh logs-check
python3 -B -m unittest discover -s team/tests -p 'test_*.py'
```

预期结果：

- `verify.py` 输出 `FINAL_PACKAGE_PASS`，交付文件哈希、`src` 源码树和 10 个子模块固定提交全部一致。
- `check` 依次输出 `VDOT_MODEL_CHECK_PASS`（参考模型 35 项测试、5 个数据文件逐字节重生成一致）和 `STATIC_SOURCE_AND_ORACLES_PASS`（32,768 种编码组合、22 个定向和 10,000 个随机向量）。

- `logs-check` 输出 `LOG_VERIFY_PASS`，9,679 个归档日志成员校验通过。
- 宿主检查器 146 项测试通过。

3. 环境准备

工具版本：GraalVM JDK 21.0.12、Mill 0.12.3、firtool 1.62.1、Verilator 5.052、riscv64-linux-gnu-gcc 11.4.0、Python 3.10+。系统软件包清单见 `submission/logs/stage1/ubuntu-packages.lock.tsv`，JDK 安装方法见第一阶段报告 1.2.2 节。

```
export TEAM_XS_ENV_ROOT=$HOME/riscv-lab/xs-env       # xs-env、NEMU、nexus-am 的安装位置
bash team/scripts/prepare-dependencies.sh            # 按版本锁检出 xs-env、NEMU、nexus-am、DRAMsim3
export JAVA_HOME=<GraalVM JDK 21.0.12 安装目录>
source team/scripts/activate.sh
bash team/scripts/check-environment.sh
export AM_HOME=$TEAM_XS_ENV_ROOT/nexus-am
```

`activate.sh` 会设置 `NOOP_HOME`（本仓库）、`NEMU_HOME`（xs-env 中的原版 NEMU）和 `TEAM_EVIDENCE_DIR`。每打开一个新终端都要重新 `source` 一次。

4. 阶段一：hello 输出与向量加法

阶段一使用原版 NEMU 和默认配置的 EMU。

```
export TEAM_EVIDENCE_DIR=$OUT/stage1
bash team/scripts/stage1.sh nemu
bash team/scripts/stage1.sh emu
TEAM_ID=<团队标识> bash team/scripts/stage1.sh workload
DUMP_WAVE=1 bash team/scripts/stage1.sh run
```

预期结果：

- 每一步最后打印 `STAGE1_ACTION_PASS`。
- `$OUT/stage1/emulator-output.log` 中有 `hello xiangshan, 我是<团队标识>, IPaddress=<本机 IPv4>`，随后是向量加法的输入与 8 个字节结果，DiffTest 无报错。
- `$OUT/stage1/run-summary.log` 是运行摘要；`$OUT/stage1/hello-vadd.fst` 是波形，用 `gtkwave $OUT/stage1/hello-vadd.fst` 打开。信号选择和提交 PC 的定位方法见第一阶段报告 1.5 节。

阶段一的 EMU 生成在仓库的 `build/` 下。最终硬件的构建器不会覆盖已有的 `build/`，所以继续之前先把它移走：

```
mv build "$OUT/stage1-build"
```

5. 构建最终硬件与 vdot NEMU

最终硬件配置：MinimalConfig、单核、VLEN=128，两槽弹性 VDot（`VDOT_O2_ELASTIC=1`）、融合输入（`VDOT_T14_FUSED_INPUT=1`），O1 关闭。

```
bash submission/reproduce.sh build-emu --output $OUT/emu-record --profile observe --firtool-dir
<firtool 1.62.1 的 bin 目录>
```

```
cp build/emu $OUT/emu-final
export VDOT_EMU_BIN=$OUT/emu-final

export TEAM_NEMU_VDOT_ROOT=$OUT/nemu-vdot
export TEAM_EVIDENCE_DIR=$OUT/nemu-build
bash team/scripts/prepare-nemu-vdot.sh reference
export NEMU_HOME=$TEAM_NEMU_VDOT_ROOT/reference
export VDOT_NEMU_REF=$NEMU_HOME/build/riscv64-nemu-interpreter-so
```

`prepare-nemu-vdot.sh` 在固定的 NEMU 提交上打 vdot 补丁（`team/patches/nemu/`），打开 `CONFIG_CUSTOM_VDOT` 等选项，在 `$TEAM_NEMU_VDOT_ROOT/reference` 下编译成 DiffTest 参考库。它在子进程中运行，所以要像上面一样显式导出 `NEMU_HOME` 和 `VDOT_NEMU_REF`，后续整核运行才会用 vdot 版 NEMU 做差分。

启动 flash 镜像，后面的整核运行都用它：

```
riscv64-linux-gnu-gcc -c -march=rv64gc -mabi=lp64d team/workloads/hello-vadd/boot-flash.S -o $OUT/
boot-flash.o
riscv64-linux-gnu-objcopy -O binary -j .text $OUT/boot-flash.o $OUT/boot-flash.bin
```

6. 阶段二：设计与基础测试

6.1 单元级 RTL 检查

```
bash team/scripts/check-vdot-datapath.sh
bash team/scripts/check-vdot-decode.sh
VDOT_T14_FUSED_INPUT=1 bash team/scripts/check-vdot-fu.sh
```

- `check-vdot-datapath.sh`：乘积与归约模块（`VDotDatapath.scala`，最终两槽 FU 也使用这两个模块）对定向与随机输入逐项比对完整 128 位结果，最后打印 `VDOT_DATAPATH_CHECK_PASS`。
- `check-vdot-decode.sh`：vdot 机器字与相邻编码的译码结果检查。
- `check-vdot-fu.sh`：基础单槽 VDot 的 FU 级握手、背压、取消、写回身份检查，并做故障注入（要求被检出），最后打印 `VDOT_FU_CHECK_PASS`。

最终两槽弹性 FU 与 VFEX4 父级（与整数除法、浮点除法/开方共用）：

```
bash team/scripts/check-vdot-o2.sh fu
bash team/scripts/check-vdot-o2.sh parent
```

脚本固定 `VDOT_O2_ELASTIC=1 VDOT_T14_FUSED_INPUT=1 VDOT_O1_HANDOFF=0`，依次生成 RTL、用 Verilator 编译测试顶层、跑全部 10022 组向量，再逐个注入故障并要求被检出：

- `fu`：打印 `O2_FU_PASS vectors=10022 ... ii1_intervals=10021`（连续独立请求每拍发起），`identity`、`wen`、`high`、`cancel` 4 种故障全部检出，最后打印 `VDOT_O2_CHECK_PASS stage=fu faults=4`。
- `parent`：打印 `O2_PARENT_PASS` 并通过逐拍事件回放，16 种故障（身份、高位、写使能、路由、时钟、占用、取消、复位等）全部检出，最后打印 `VDOT_O2_CHECK_PASS stage=parent faults=16`。
- 2026-11-08 在最终源码上的运行日志：`submission/verification/o2-check-20261108/`。

6.2 整核 vdot 测试（正数、负数、边界、异常）

```
TEAM_EVIDENCE_DIR=$OUT/vdot-host  bash team/scripts/vdot-tests.sh host
TEAM_EVIDENCE_DIR=$OUT/vdot-build bash team/scripts/vdot-tests.sh build-xs full
```

```
TEAM_EVIDENCE_DIR=$OUT/vdot-rtl VDOT_IMAGE=$OUT/vdot-build/vdot-exceptions.bin \
  bash team/scripts/vdot-tests.sh run-rtl full
```

- `host` 和 `build-xs` 会先用参考模型生成测试数据（`team/out/t05`），其中包括赛题 2.3.1 ~ 2.3.3 的正数、负数、边界用例和非法配置的异常用例。
- `run-rtl` 在最终 EMU 上运行测试程序，并以 vdot NEMU 做 DiffTest 逐条比对。检查器 `check-vdot-run.py --require-diff` 通过时打印 `VDOT_WORKLOAD_PASS`；运行日志在 `$OUT/vdot-rtl/run.log`。
- 第二阶段报告中三个基础用例的结果分别为 204、–8、131072，写回的高 96 位均为零。

7. 阶段三：协同仿真与性能分析

7.1 微基准（VDot 与等价 RVV 同镜像对比）

四个场景（单次点积、256 次依赖链、128 次四路独立批量、64 次加载-计算-存储流）各用三个数据集、每个数据集四个正式窗口，在最终 EMU 上运行，周期由程序内 `cycle` CSR 读数得到，加速比为同场景 RVV 平均周期除以 VDot 平均周期。全部原始运行日志在证明材料 `提交/测试数据与运行结果证明材料/evidence/`，用下面的命令从已保存的逐窗口结果重新计算表 3-3 和表 3-4：

```
cd 提交/测试数据与运行结果证明材料
python3 tools/prepare_data.py          # 输出 microbench 与 c8 两组汇总
git diff --stat data/微基准结果汇总.csv data/C8最终尺寸汇总.csv    # 无差异，即与提交的数据表一致
git checkout -- data                   # 恢复运行时间等字段
cd -
```

输出与报告表 3-3 一致：

场景	VDot 周期	RVV 周期	加速比
单次点积	71	85.25	1.2007×
依赖链（256 次）	1857	5440.5	2.9297×
四路独立批量（128 次）	221	678	3.0679×
加载、计算、存储流（64 次）	1127.5	1133	1.0049×

C8 一组输出为 firstdot V2 282.25、C8 273、最快 RVV 299 周期（表 3-4）。

在新构建的硬件上重新测量：使用 `python3 -B team/workloads/unified-vdot/retest.py build/run/analyze`，完整命令见 性能重测说明。先运行 smoke 快速检查，再运行 full；full 要求第 5 部分的 observe-profile EMU 构建回执。程序支持新的 EMU 哈希与构建提交，保留实际运行日志并严格检查 DiffTest、退休指令和 CSR 计数窗口。历史实验程序仅作为来源记录。

7.2 软件库：firstdot V2 与 C8

```
bash submission/reproduce.sh firstdot $OUT/firstdot-sdk
bash submission/reproduce.sh c8 $OUT/c8-sdk
sha256sum $OUT/c8-sdk/gemv.o
```

两套库都提供普通 C 接口，示例程序在输出目录中编译和链接。C8 的 Makefile 会核对源文件和生成对象的固定哈希，`gemv.o` 的 SHA-256 应为 `f8bc13daeead98a414f313600a5778c96ac95a66b5bcb5c714ea3b98e0d25ab3`。8×32

GEMV 调用的周期见报告表 3-4：C8 为 273 周期，比较快的 RVV 实现为 299 周期。

7.3 vdot 波形

`build-emu` 生成的最终 EMU 不带波形输出。看 vdot 波形时，用同样的配置另外构建一份带 FST 的 EMU：

```
mv build $OUT/final-build
export VDOT_O2_ELASTIC=1 VDOT_T14_FUSED_INPUT=1 VDOT_O1_HANDOFF=0
make emu CONFIG=MinimalConfig NUM_CORES=1 EMU_THREADS=1 EMU_TRACE=fst JVM_XMX=12G -j2
./build/emu -i $OUT/vdot-build/vdot-exceptions.bin --flash=$OUT/boot-flash.bin --ram-size=128MB \
  --diff=$VDOT_NEMU_REF --max-cycles=5000000 --dump-wave --wave-path=$OUT/vdot.fst -b 0 -e 2000000
gtkwave $OUT/vdot.fst
```

在 GTKWave 中搜索 `Vdot` 实例，观察输入 valid/ready、两个源操作数、ROB 编号与物理目的、两个结果槽和写回；提交在 ROB 中确认。信号的具体含义和截图见第三阶段报告 3.4 节。

7.4 面积与时序

报告第 3.9 节的综合约束、路径报告和输入版本保存在 `submission/reports/evidence/nangate45-4ns-20261108/` 与 `sp1-4ns-comparison-20261108/`，可直接查看原始 `.rpt` 文件。

8. 原始日志

报告中各项数据对应的原始运行日志（含失败记录）按原路径打包在 `submission/logs/`：

```
python3 -B submission/logs/read_log.py list --contains stage1
python3 -B submission/logs/read_log.py show <日志路径> --head 40
python3 -B submission/logs/read_log.py restore --to $OUT/log-view
```